

DSParkCode 项目初期调研报告

阮治诚 2025020903010 唐启明 2025020904017 韩志文 2025020905010

一、项目名称

DSParkCode，是基于 DeepSeek 大语言模型构建的智能对话机器人框架。

二、选题意义

随着 Transformer 架构的提出，语言模型迎来了井喷式的发展，从最开始只能进行简单聊天到可以向人类一样理解文字，进行文字推理，再到现在不仅可以看懂文字，甚至可以解析图片、音乐与视频。随着大模型自身的能力不断进化发展，Agent（智能体）应用便出现了。如果把大模型比作人类的大脑，一个好的 Agent 框架就是人类发达的四肢。它让 AI 从一个只可以推理思考的大脑转变为一个可以真正执行任务的助手。

AI 巨头企业 Anthropic 和 OpenAI 都推出了各自的 Agent 应用如 Claude Code、Codex。这些应用受到了来自全球各地的程序员的喜爱。但对于国内用户，使用这类工具往往要面对国外模型的高额价格成本和服务商对中国大陆的严密封锁。

DSParkCode 就是来解决这些问题的——它专为国内顶尖开源大模型 DeepSeek 而生，同时主打通用用途和零门槛、轻量级部署。解决主流 Agent 门槛高，用户群体范围窄的问题。

通过这个项目，我们可以更加深入了解大语言模型的工作原理，Agent 应用的基础框架构建，同时明白 AI 不仅仅是一个聊天软件，而是一次通用技术的技术革命。

三、功能规划

（一）Function Calling 工具系统

这是整个项目的核心，是 Agent 的四肢，可以极大拓展 AI 能力边界。AI 在回复你之前，会先判断"我需要什么工具来完成这个任务"，然后自动调用对应的工具。目前计划实现 20+ 个工具，分为四大类：

文件操作类（8 个）：

工具名称	功能说明
read_file	读取 workspace 下的文件

write_file	覆盖写入文件
append_file	追加内容到文件末尾
delete_file	删除文件（需要确认）
list_files	列出目录内容
search_files	在文件中搜索关键词
move_file	移动或重命名文件
create_directory	创建文件夹

记忆管理类（4 个）：

工具名称	功能说明
add_user_memory	记录用户偏好信息
list_user_memories	查看用户画像
delete_user_memory	删除指定记忆
clear_user_memories	清空全部记忆

技能系统类（5 个）：

工具名称	功能说明
list_skills	查看可用技能列表
activate_skill	激活指定技能
deactivate_skill	关闭指定技能
read_skill_file	读取技能内容
write_skill_file	创建或修改技能

外部交互类（7 个）：

工具名称	功能说明
web_search	通过 Bing 联网搜索
tieba_get_threads	浏览贴吧帖子列表
tieba_get_posts	查看帖子详情和回复
tieba_search_exact	在指定贴吧内搜索
tieba_get_forum_info	查看贴吧信息
tieba_get_user_info	查看贴吧用户信息
tieba_get_hot_threads	查看贴吧热门帖子

（二）智能对话管理

上下文压缩：对话超过 15 轮时自动触发，将最早的 5 轮对话用 AI 压缩成摘要，防止上下文腐烂，保证 AI 回复质量，既不丢失上下文，又节省 Token 消耗。

用户画像系统：AI 自动从对话中提取你的信息（偏好、习惯、情绪），记录下来并在后续对话中参考。

可定制提示词：通过 `data/SOUL.json`（角色设定）、`data/RULES.json`（行为准则）、`data/CAPABILITY.json`（能力描述）深度定制 AI 行为。

（三）技能系统

技能是存放在 `skills/<技能名>/SKILL.md` 的 Markdown 文件，用自然语言描述"当激活这个技能时，AI 应该如何表现"。

工作机制：

1. 启动时，只加载技能名称和描述（节省 Token）
2. AI 判断需要某个技能时，调用 `activate_skill` 激活
3. 激活后，技能的完整行为指令被注入系统提示词
4. 最多同时激活 3 个技能，超出自动关闭最早的
5. AI 可以自己创建新技能（调用 `write_skill_file`）

（四）Web UI / 图形界面

不只是局限于命令行工具，通过现有的 UI 框架可以为项目应用开发跨平台的、美观的用户交互界面，让任务执行不再是一行行繁杂的控制台输出，而是可视可控的图形化输出。

（五）知识库系统

构建本地知识库，并利用 RAG 技术检索本地知识库，获取定制化内容。

四、案例参考

Claude Code 的 Skill 系统

来源：Anthropic 公司开发的 AI 编程助手（Claude Code CLI）

借鉴了什么：

技能作为 Markdown 文件的渐进式加载机制是 Claude Code 的核心设计。目录层：只加载技能名称和简短描述，几乎不消耗 Token。激活层：用户或 AI 激活某个技

能后，才加载完整的 SKILL.md 内容。管理层：AI 可以自己创建、修改、删除技能，实现自我进化。这个设计非常巧妙——如果一开始就把所有技能内容塞进提示词，Token 消耗巨大，对话几轮就没额度了。渐进式加载既省钱又灵活。

为什么参考它：Claude Code 的 Skill 系统是目前业界最成熟的 AI 技能管理方案，经过了大规模实际使用验证，设计思路值得学习。

Hermes 项目的用户画像与技能自更新

来源：GitHub 高星开源 Agent Hermes

借鉴了用户画像自动挖掘机制：AI 主动从对话中推断用户偏好（比如用户说"今天打了一下午游戏"，AI 记录"用户喜欢玩游戏"），而不是被动等用户来填写资料。以及技能自我创建：AI 通过 write_skill_file 工具自主创造新技能——用户说"下次遇到这种情况你就这样做"，AI 就自动把这个行为保存为技能。

为什么参考它：Hermes 是经过实际使用打磨的 GitHub 热门项目，用户画像和技能自更新的功能已经被验证是成熟的。

五、技术栈分析

异步 HTTP + SSE 流解析：用异步在不阻塞主线程的情况下发送 API 请求，并对 AI 回复进行流式输出。

大语言模型云服务：对接 DeepSeek 供应商 API，完整正确构建并发送请求体并解析返回的数据。

外部功能性 API 封装：对接如 B 站、贴吧的官方 API，赋予 AI 如获取 B 站视频、爬取贴吧帖子等扩展内容。

结构化数据处理：将用户数据、对话历史转换为 JSON 结构数据处理，将 AI 输出的 JSON 结构数据解析为自然语言。

预期核心项目文件结构

文件路径	说明
boot.py	一键安装配置脚本
pyproject.toml	项目元数据 + 依赖声明
data/UserSettings.json	API Key + 配置
data/SOUL.json	AI 角色设定（可自定义）
data/RULES.json	AI 行为准则（可自定义）
data/CAPABILITY.json	AI 能力描述

data/MemoryForUser.json	用户画像（AI 自动生成）
data/ConversationHistory.json	对话记录（自动保存）
workspace/	文件操作沙箱
skills/	技能目录
src/atri/main.py	入口程序（控制台 UI）
src/atri/ai_service.py	核心对话引擎（API 通信 + Function Calling 循环）
src/atri/tool_manager.py	工具注册与执行分发
src/atri/file_tool.py	文件操作工具实现
src/atri/memory_tool.py	用户记忆工具实现
src/atri/skill_loader.py	技能加载器
src/atri/tieba_tool.py	百度贴吧工具实现
src/atri/prompt_manager.py	系统提示词组装
src/atri/conversation.py	对话历史管理
src/atri/content_compact.py	上下文自动压缩
src/atri/setup.py	配置向导
src/atri/models.py	数据模型定义
tests/test_ai_service.py	AI 服务测试
tests/test_tool_manager.py	工具管理测试
tests/test_file_tool.py	文件工具测试
tests/test_memory_tool.py	记忆工具测试
tests/test_conversation.py	对话管理测试

六、小组分工建议

前端交互（1 人）

负责内容：用户交互界面构建开发；用户输入处理与指令系统（/help、/status、/clear、/exit）；流式输出的显示优化；配置向导（atri-setup）的交互体验。

需要掌握：Python 基础、简单前端开发 UI 框架。

后端核心（1 人）

负责内容：DeepSeek API 通信模块（httpx 异步请求 + SSE 流解析）；Function Calling 工具调度引擎（Tool Loop 循环逻辑）；技能系统核心（SkillLoader：渐进式加载、激活/关闭、自创建）；用户画像与记忆管理（MemoryTool）；上下文压缩算法（ContentCompact）。

需要掌握：Python 异步编程（async/await）、HTTP 协议基础、JSON 数据处理。

测试与文档（1 人）

负责内容：单元测试编写（`pytest + pytest-asyncio`）；工具调用功能验证（`FileTool`、`MemoryTool`、`SkillLoader`）；API 通信异常测试（网络断开、API 返回错误、超时等）；技能系统边界条件测试（激活超限、技能不存在、格式错误等）；项目文档维护（`README`、用户手册、API 文档）。

需要掌握：`pytest` 测试框架、Python 基础。

七、总结

`DeepSeekParkCode` 是一个定位清晰、架构简洁、可玩性高的 **Agent** 项目。它最大的特点就是“麻雀虽小，五脏俱全”——只有两个第三方依赖，但完整实现了 `Function Calling`、用户画像、技能系统、上下文压缩等企业级功能。